

### Problem Statement:

In a network environment, a shared printer receives multiple print requests from users throughout the day. To handle these requests efficiently and use optimal memory usage, the

system implements a linear queue to manage the list of print jobs. Each print job is identified

by a Job ID and a Document title. The print jobs are processed in the order they are received.

(FIFO order), and memory is released as jobs are completed.

### Objective:

- 1) Implement a linear queue using an array.
- 2) Perform enqueue, Dequeue and Display operations
- 3) Understand FIFO behavior and overflow/underflow conditions

### Theory:

A queue is a linear data structure that follows the FIFO (First In, First out) principle. This means the element inserted first is the first out to be removed, similar to a queue of people waiting for a bus or at a ticket counter.

In a queue

- Insertion is performed at the rear end
- Deletion is performed at the front end.

## Types of queue

### 1) Simple Queue

Follows FIFO.

- Insertion at rear.
- Deletion at front.

### 2) Circular Queue

- Last position connects to the first position.
- Efficiently utilizes array space by reusing free positions.

### 3) Priority Queue

- Elements are removed based on priority rather than insertion order.
- Higher-priority elements are served first.

### 4) Double-Ended Queue (Deque)

- Insertion and deletion are allowed at both the front and rear.

#### Types

- Input-restricted Deque (insertion at one end only)
- Output-restricted Deque (deletion at one end only)

## Application of queue

- 1) CPU process scheduling
- 2) Printer spooling.
- 3) Breadth-First Search (BFS) in graphs

4. Resource sharing
5. Network packet buffering.
6. Custom process systems
7. Interrupt handling in operating systems.
8. Simulation of real world waiting lines.

## Implementation

### platform:

- C++ bit open source library or its derivatives
- open source C programming tool like gcc / Eclipse Editor.

## Test conditions

| Test conditions | Input                                             | Expected output                                                               |
|-----------------|---------------------------------------------------|-------------------------------------------------------------------------------|
| TC01            | Create empty queue                                | Queue initialized successfully                                                |
| TC02            | Enqueue job ID: 101<br>Report pdf                 | print job added successfully                                                  |
| TC03            | Enqueue print jobs until<br>queue is full         | All jobs inserted successfully<br>until the queue reaches<br>maximum capacity |
| TC04            | Display Enqueue print jobs<br>until queue is full | Queue overflow! Print<br>job is full.                                         |
| TC05            | Display queue                                     | All pending print jobs is<br>not removed.                                     |
| TC06            | Dequeue one print job                             | The first inserted print job is<br>processed removed                          |
| TC07            | Dequeue all print jobs                            | All jobs are processed one by<br>one until the queue becomes<br>empty.        |
| TC08            | Dequeue from an empty queue                       | Queue underflow! No print<br>jobs available                                   |

TC 09 Display on empty queue

Queue underflow, No print jobs available

TC 10 Exit the program

Program terminates successfully.

## • Pseudocode

### Initialization

Initialize front = -1

Initialize rear = -1

Create an array Queue

## • Enqueue

→ Enqueue (job)

If rear == max - 1

Print "Queue overflow Print queue is full"

Return

If front == -1

front = 0

rear = rear + 1

Queue[rear] = job

Print "Print job added successfully"

## Dequeue

Dequeue()

If  $\text{front} == -1$  or  $\text{front} > \text{rear}$

Print "Queue Underflow! no print jobs available"

Return

$\text{job} = \text{queue}[\text{front}]$

Print "processing job"

$\text{front} = \text{front} + 1$

If  $\text{front} > \text{rear}$

$\text{front} = -1$

$\text{rear} = -1$

• Display

Display()

If  $\text{front} == -1$

Print "No pending Print Jobs"

Return

For  $i = \text{front}$  to  $\text{rear}$

Display  $\text{QUEUE}(i)$

Overflow checking

If  $\text{rear} == \text{max} - 1$

Queue is full

Underflow checking

If  $\text{front} == -1$

or  $\text{front} > \text{rear}$   
Queue is Empty

## • Time Complexity

| operations       | Time Complexity |
|------------------|-----------------|
| Initialization   | $O(1)$          |
| Enqueue          | $O(1)$          |
| Dequeue          | $O(1)$          |
| Display          | $O(n)$          |
| overflow check   | $O(1)$          |
| underflow check. | $O(1)$          |

## • Conclusion:

- The printer queue manager using a linear queue successfully implements FIFO scheduling for print jobs. It performs Enqueue, Dequeue, and display operations correctly while handling overflow and underflow conditions, demonstrating the practical application of the linear queue data structure in managing print requests.

## • FAQs

1) What is a linear ~~search~~ queue?

Ans: A linear queue is a linear data structure that follows the FIFO (First In First Out) principle. Insertion takes place at the rear and deletion takes place at the front.

2) What principle does a linear queue follow?

→ It follows the FIFO - First In, First Out principle.

Ex:

Insert:  $101 \rightarrow 102 \rightarrow 103$

Remove:  $101 \rightarrow 102 \rightarrow 103$

3) What are the basic operations?

→ The basic operations are:

- Enqueue → Insertion
- Dequeue → deletion
- Display → display queue elements.

4) What is enqueue?

Ans Enqueue is the operation of inserting a new element at the rear of the queue.

5) What is Dequeue?

Ans Dequeue is the operation of removing an element from the front of the queue.

6) How does a linear queue reuse memory?

Ans A standard queue implemented with an array has a limitation. After elements are dequeued, their earlier occupied positions are not ~~to~~ reused by a linear enqueue operation. A circular queue is specifically designed to reuse those freed positions.

7) What are applications of a linear queue?

Ans Application include

• Printer spooling

• CPU process scheduling.

• DFS

• Network packet buffering

• Resource sharing

• Customer service systems.

• Interrupt handling.

8) what is the time complexity of Enqueue and Dequeue

Ans For this array implementation

→ Enqueue =  $O(1)$

→ Dequeue =  $O(1)$

9) what is the limitation of linear queue

Ans The main limitation is wasted array space. When elements are removed from the queue positions cannot be normally be used by simple linear queue, even though there may be free space at the beginning a circular queue address this issue